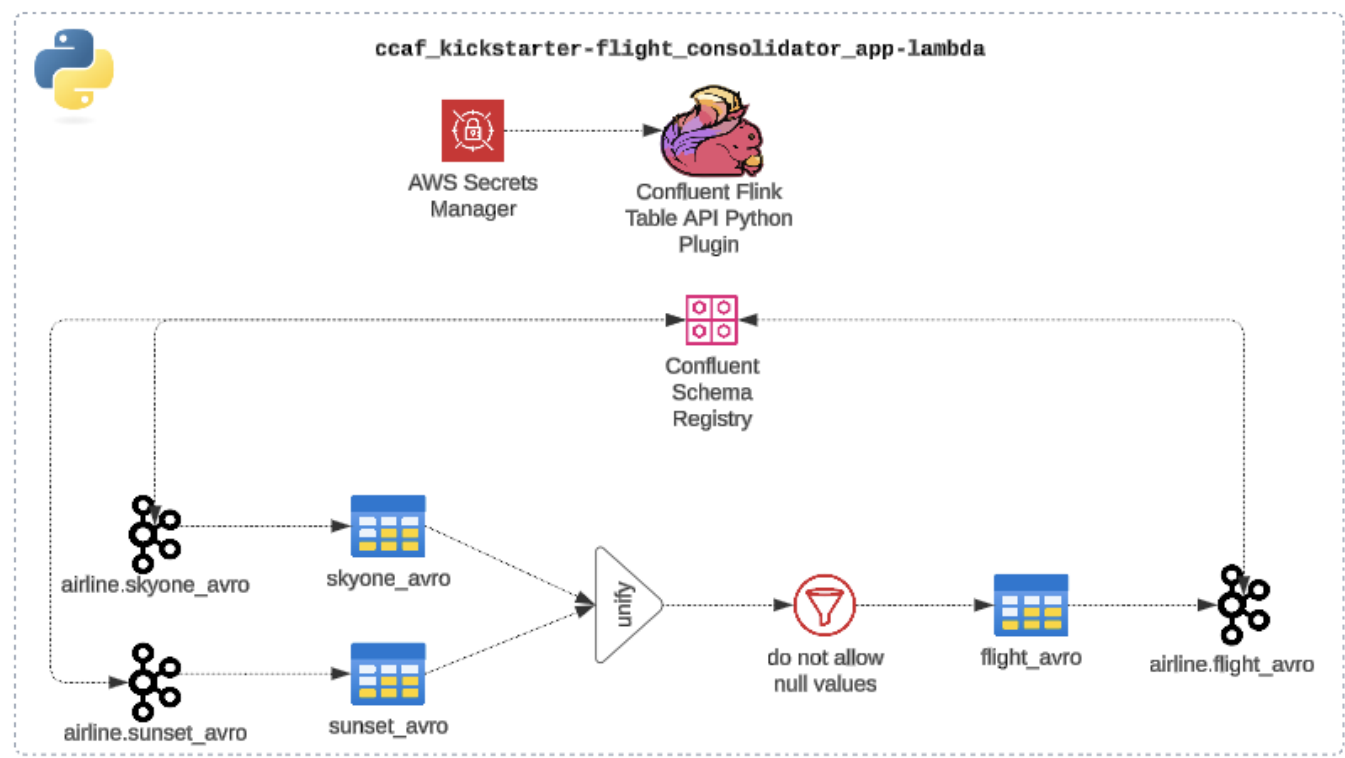


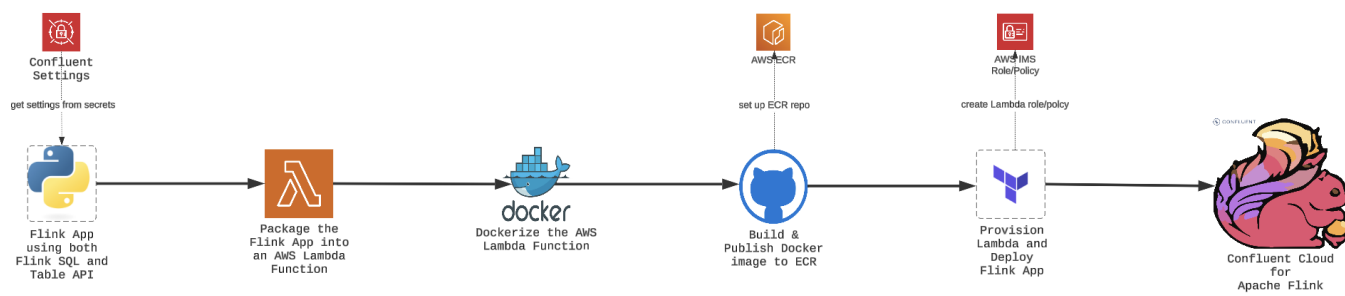
Confluent Cloud for Apache Flink (CCAF) Flight Consolidator App Lambda

This repository empowers the deployment of a robust Flink Application to [Confluent Cloud for Apache Flink \(CCAF\)](#) , enabling seamless and continuous real-time streaming. The Flink Application meticulously ingests flight data from the `airline.sunset_avro` and `airline.skyone_avro` Kafka topics, standardizing and unifying the information into a single, consolidated `airline.flight_avro` Kafka topic. By leveraging advanced stream processing capabilities, this deployment ensures high scalability, data consistency, and reliability, providing organizations with a powerful foundation for actionable flight analytics and insightful decision-making.

Architectural Drawing of the Flink App



Deployment Flow of the Flink App



Note: Unlike the open-source Apache Flink, Confluent Cloud for the Apache Flink Table API generates SQL rather than the low-level `DataStream` API code found in open-source Apache Flink. This SQL is ultimately deployed to Confluent Cloud for Apache Flink! The Confluent Flink Table API Python Plugin accomplishes this by converting the code into SQL. Therefore, remember that your

code does not execute in Confluent Cloud for Apache Flink; it is simply translated into SQL. This is why this Lambda deployment flow is recommended, as it streamlines the SQL deployment process.

Table of Contents

- **1.0 Let's get started!**
 - **1.1 Use the Bash script supplied in the repo to deploy the Lambda function from your local machine**
 - **1.1.1 Clone the repo**
 - **1.1.2 Navigate to the Root Directory**
 - **1.1.3 Launch from your local machine the supplied Bash script to Publish (Create) or Unpublish (Delete) the Lambda Function**
 - **1.2. Use GitHub to launch the Lambda function from the cloud**
 - **1.2.1 Deploy the Repository**
 - **1.2.2 Set Required Secrets and Variables**
 - **1.2.3 Navigate to the Actions Page**
 - **1.2.4 Select and Run the Deploy Workflow**
- **2.0 Resources**

Please run the **AvroDataGeneratorApp** first to generate the **airline.sunset_avro** and **airline.skyone_avro** Kafka topics before running this application.

1.0 Let's get started!

Take care of the cloud and local environment prerequisites listed below:

1. You need to have the following cloud accounts:
 - [AWS Account](#) with SSO configured
 - [Confluent Cloud Account](#)
 - [Docker Account](#)
 - [GitHub Account](#) with OIDC configured for AWS
 - [Terraform Cloud Account](#)
2. You need to have the following installed on your local machine:
 - [AWS CLI version 2](#)
 - [aws2-wrap](#) utility
 - [Confluent CLI version 3 or higher](#)
 - [Docker Desktop](#)
 - [Java JDK \(Java Development Kit\) 11](#)
 - [Python 3.11](#)
 - [Terraform CLI version 1.9.3 or higher](#)

1.1 Use the Bash script supplied in the repo to deploy the Lambda function from your local machine

1.1.1 Clone the repo

```
git clone https://github.com/j3-signalroom/ccaf-kickstarter-flight-consolidator_app-lambda.git
```

1.1.2 Navigate to the Root Directory

Open your Terminal and navigate to the root folder of the `ccaf-kickstarter-flight-consolidator_app-lambda/` repository that you have cloned. You can do this by executing:

```
cd path/to/ccaf-kickstarter-flight-consolidator_app-lambda/
```

Replace `path/to/` with the actual path where your repository is located.

1.1.3 Launch from your local machine the supplied Bash script to Publish (Create) or Unpublish (Delete) the Lambda Function

Execute the `run-local.sh` script to:

- create an AWS Elastic Container Registry (ECR) repository,
- build the AWS Lambda Docker container,
- publish it to the newly created ECR repository,
- and then run the [Terraform configuration](#) to either create or destroy the IAM Policy and Role for the Lambda, and
- invoke (run) the Lambda.

Use the following command format:

```
scripts/run-local.sh <create | delete> --profile=<SSO_PROFILE_NAME> --catalog-name=<CATALOG_NAME> --database-name=<DATABASE_NAME> --ccaf-secrets-path=<CCAF_SECRETS_PATH>
```

Replace Argument Placeholders

- `<create | delete>`: Specify either `create` to create the ECR repository or `delete` to remove it.
- `<SSO_PROFILE_NAME>`: Replace this with your AWS Single Sign-On (SSO) profile name, which identifies your hosted AWS infrastructure.
- `<CATALOG_NAME>`: Replace this with the name of your Flink catalog.
- `<DATABASE_NAME>`: Replace this with the name of your Flink database.
- `<CCAF_SECRETS_PATH>`: Replace this with the path to the Confluent Cloud for Apache Flink (CCAF) AWS Secrets Manager secrets.

For example, to publish the Lambda from your local machine, use the following command:

```
scripts/run-local.sh create --profile=my-aws-sso-profile --catalog-  
name=flink_kickstarter --database-name=flink_kickstarter --ccaf-secrets-  
path="/confluent_cloud_resource/flink_kickstarter/flink_compute_pool"
```

Replace `my-aws-sso-profile` with your actual AWS SSO profile name, `flink_kickstart` Flink catalog (Environment), `flink_kickstart` Flink database (Kafka Cluster) and the `/confluent_cloud_resource/flink_kickstarter/flink_compute_pool` AWS Secrets Manager secrets path.

1.2. Use GitHub to launch the Lambda function from the cloud

1.2.1 Deploy the Repository

Ensure that you have cloned or forked the repository to your GitHub account.

1.2.2 Set Required Secrets and Variables

Before running any of the GitHub workflows provided in the repository, you must define at least the `AWS_DEV_ACCOUNT_ID` variable (which should contain your AWS Account ID for your development environment). To do this:

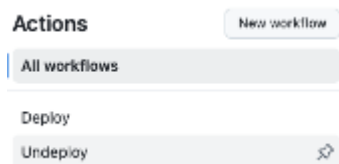
- Go to the Settings of your cloned or forked repository in GitHub.
- Navigate to **Secrets and Variables > Actions**.
- Add the `AWS_DEV_ACCOUNT_ID` and any other required variables or secrets.

1.2.3 Navigate to the Actions Page

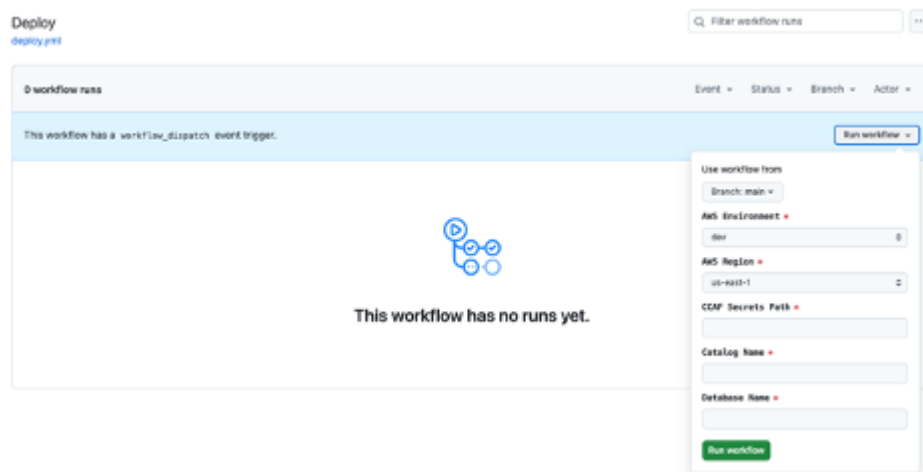
- From the cloned or forked repository on GitHub, click on the **Actions tab**.

1.2.4 Select and Run the Deploy Workflow

- Find the **Deploy** workflow link on the left side of the **Actions** page and click on it.



- On the **Deploy** workflow page, click the **Run workflow** button.
- A workflow dialog box will appear. Fill in the necessary details and click **Run workflow** to initiate the building and publishing the Lambda docker container to ECR, and using Terraform to create the IAM Policy and Role for the Lambda and then invoke (run) the Lambda.



Upon completing the steps outlined above, you will have successfully deployed the **Flight Consolidator Flink Application** to your [Confluent Cloud for Apache Flink \(CCAF\)](#) environment. This deployment not only exemplifies industry-leading best practices for managing Flink Application but also harnesses the full power and seamless integration of Confluent Cloud for Apache Flink. Empower your data processing workflows with unparalleled efficiency, scalability, and reliability through this cutting-edge demonstration.

2.0 Resources

[Confluent Cloud for Apache Flink: Best Practices for Deploying Table API Applications with GitHub and Terraform](#)

[Flink Applications Powered by Python on Confluent Cloud for Apache Flink \(CCAF\)](#)

[Confluent Cloud for Apache Flink \(CCAF\) Documentation](#)

[Apache Flink Kickstarter](#)